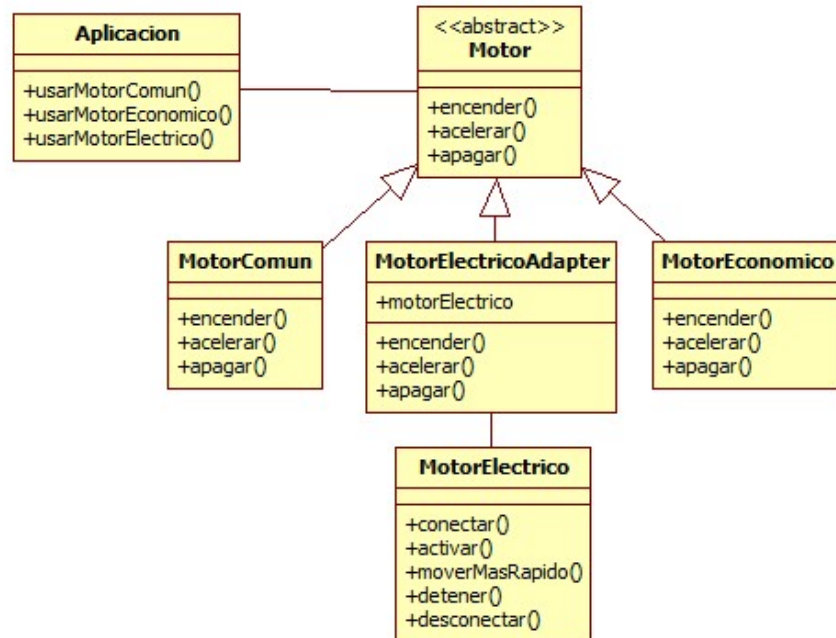
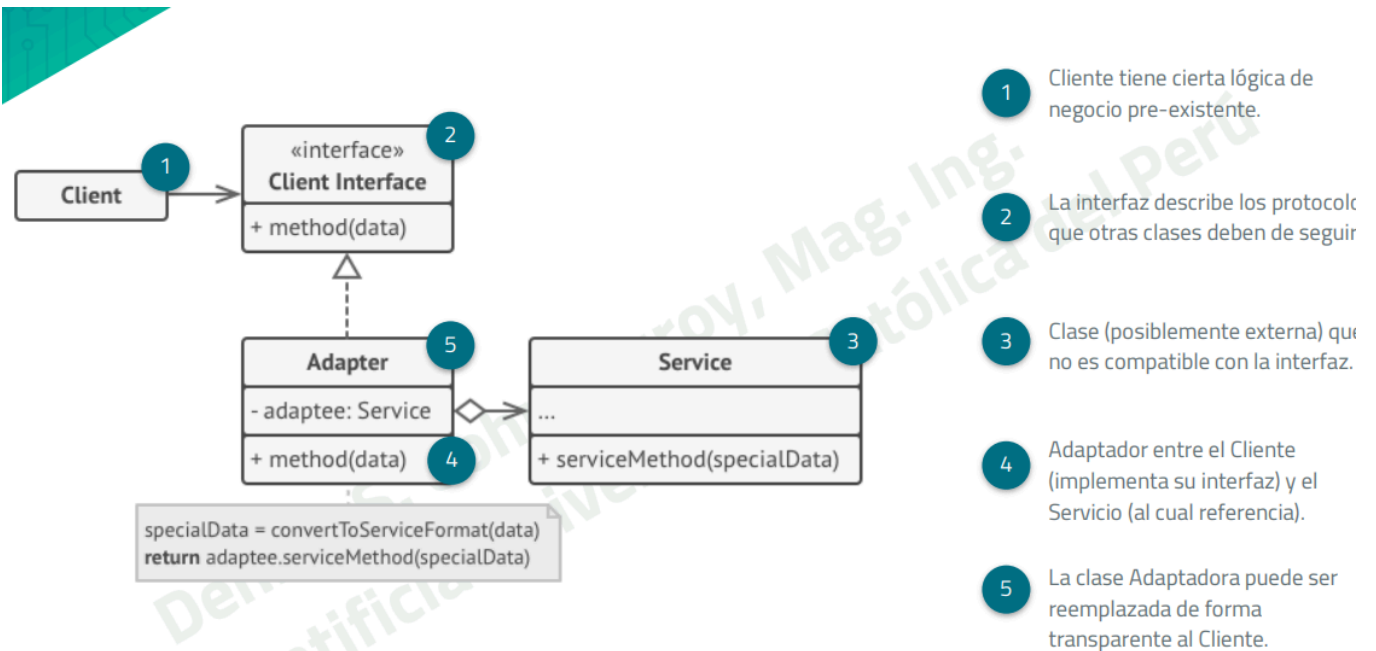
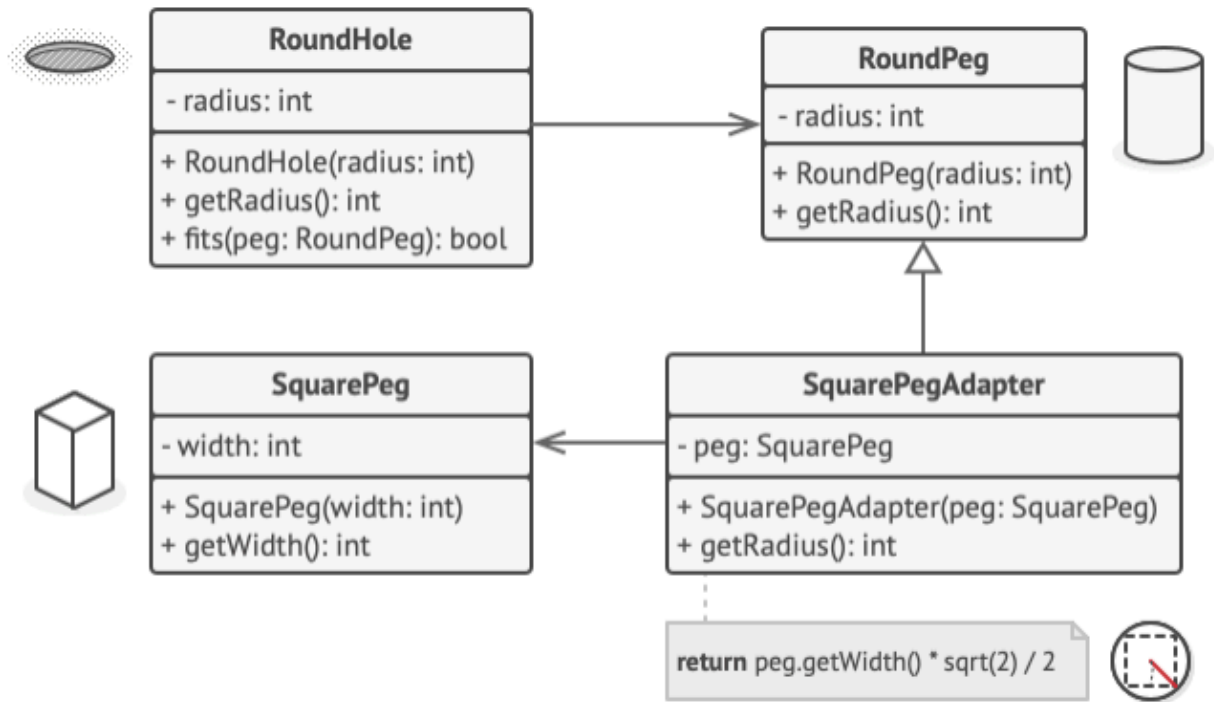


1. Adapter (Adaptador)





Problema que resuelve: Permitir que objetos con interfaces incompatibles trabajen juntos.

Cómo funciona: Crea una clase intermediaria que "traduce" entre dos interfaces incompatibles.

Ejemplo:

```

java
// Servicio externo con interfaz incompatible
class ServicioExterno {
    public void metodoEspecial(String datoEspecial) { ... }
}

// Adaptador que hace compatible nuestro sistema
class Adaptador implements MiInterfaz {
    private ServicioExterno servicio;

    public void metodoComun(String dato) {
        String datoConvertido = convertirDato(dato);
        servicio.metodoEspecial(datoConvertido);
    }
}

// Cliente usa la interfaz normal
MiInterfaz objeto = new Adaptador(servicioExterno);
  
```

```
objeto.metodoComun("dato");
```

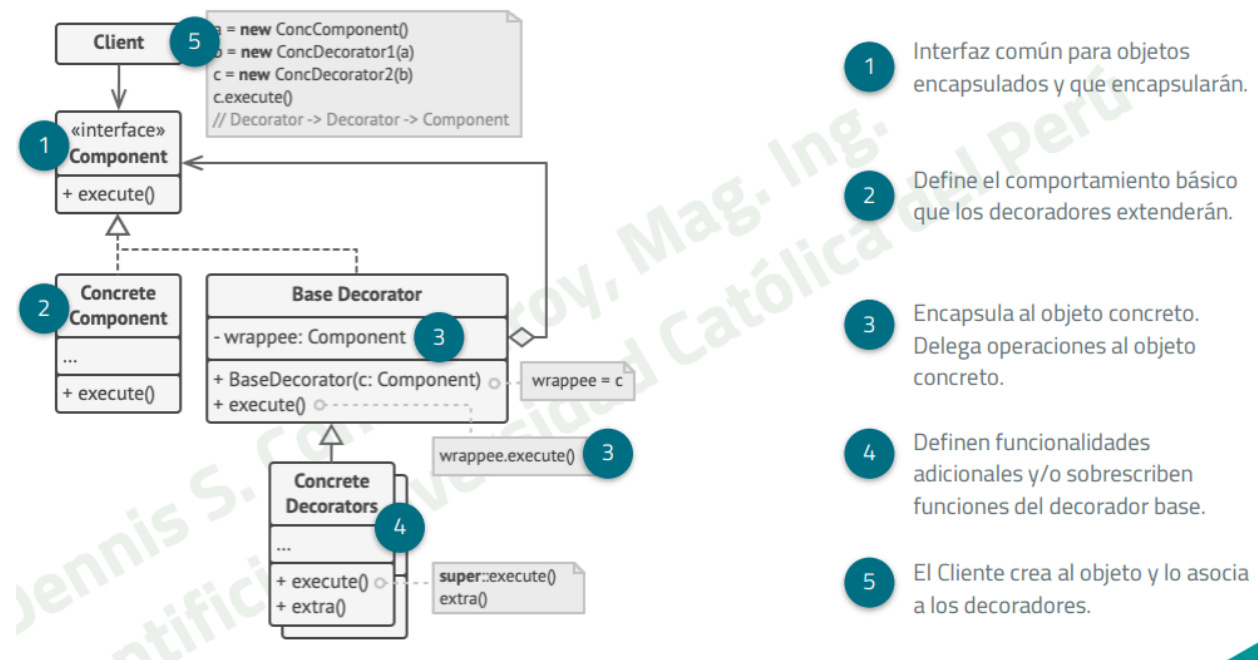
Cuándo usarlo:

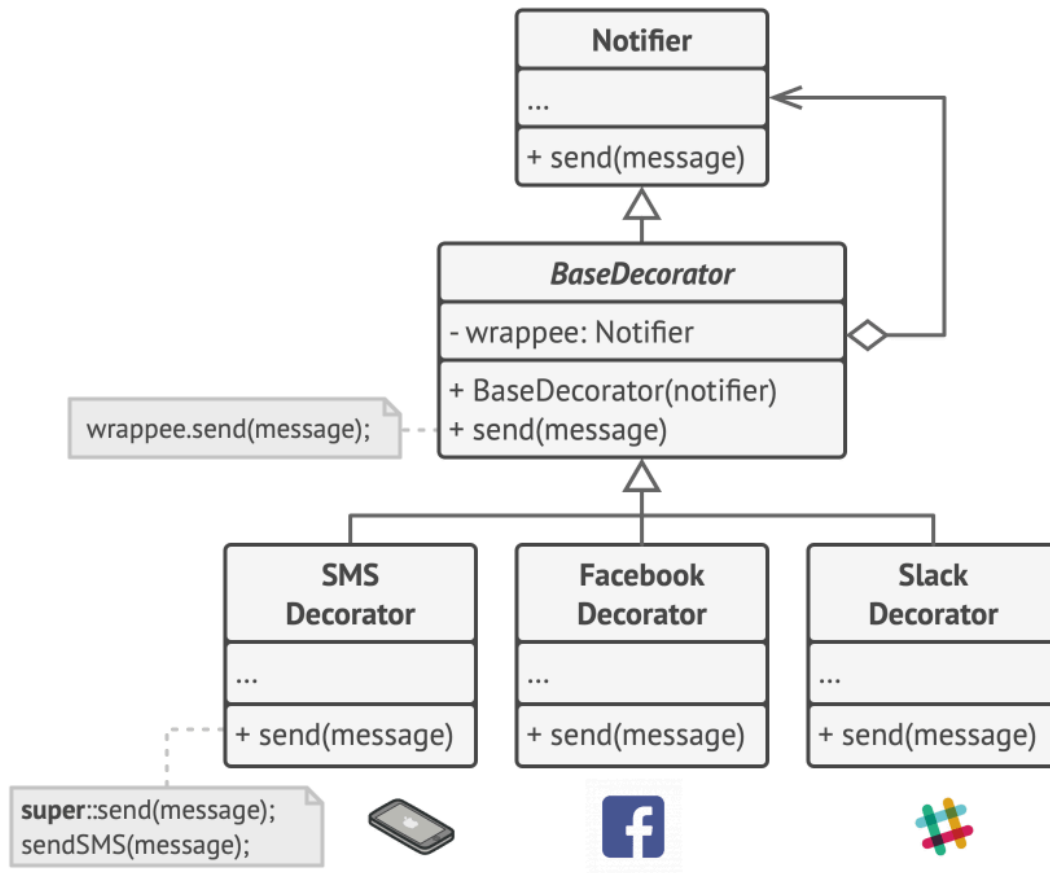
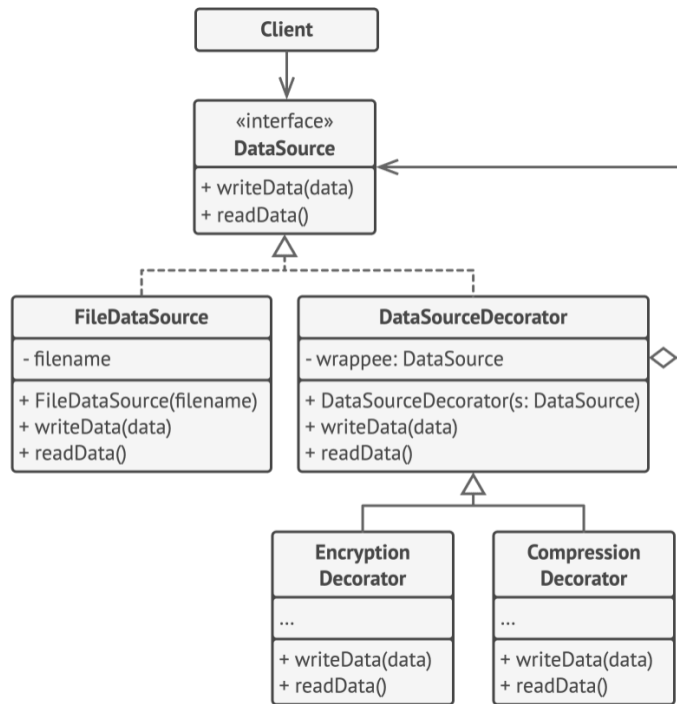
- ☒ Integrar clases legacy o externas
- ☒ Hacer compatible código de diferentes equipos
- ☒ Reutilizar clases existentes con interfaces diferentes

Consecuencias:

- **+** Separa conversión de lógica de negocio
- **+** Transparente para el cliente
- **-** Aumenta complejidad con nuevas interfaces

2. Decorator (Decorador)





Problema que resuelve: Añadir responsabilidades a objetos individuales de forma dinámica.

Cómo funciona: Envuelve objetos en "wrappers" que añaden funcionalidad.

Ejemplo:

```
java

// Componente base
interface Cafe {
    double costo();
    String descripcion();
}

// Implementación concreta
class CafeSimple implements Cafe {
    public double costo() { return 5.0; }
    public String descripcion() { return "Cafe simple"; }
}

// Decoradores
class ConLeche extends DecoradorCafe {
    public ConLeche(Cafe cafe) { super(cafe); }
    public double costo() { return super.costo() + 2.0; }
    public String descripcion() { return super.descripcion() + ", leche"; }
}

class ConChocolate extends DecoradorCafe {
    public ConChocolate(Cafe cafe) { super(cafe); }
    public double costo() { return super.costo() + 3.0; }
    public String descripcion() { return super.descripcion() + ", chocolate"; }
}

// Uso
Cafe miCafe = new CafeSimple();
miCafe = new ConLeche(miCafe);
miCafe = new ConChocolate(miCafe);

// Resultado: "Cafe simple, leche, chocolate" - costo: 10.0
```

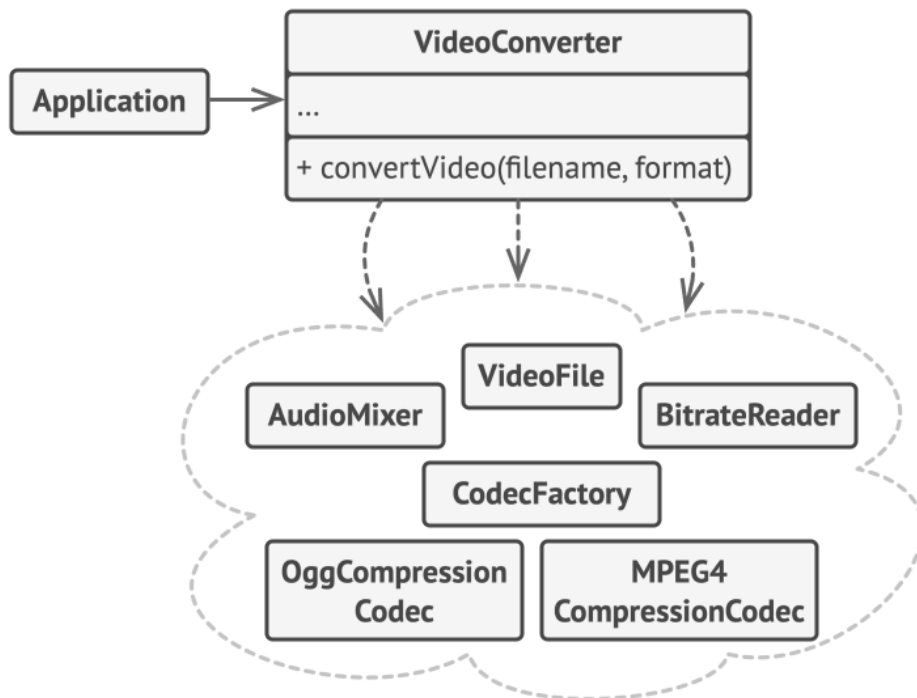
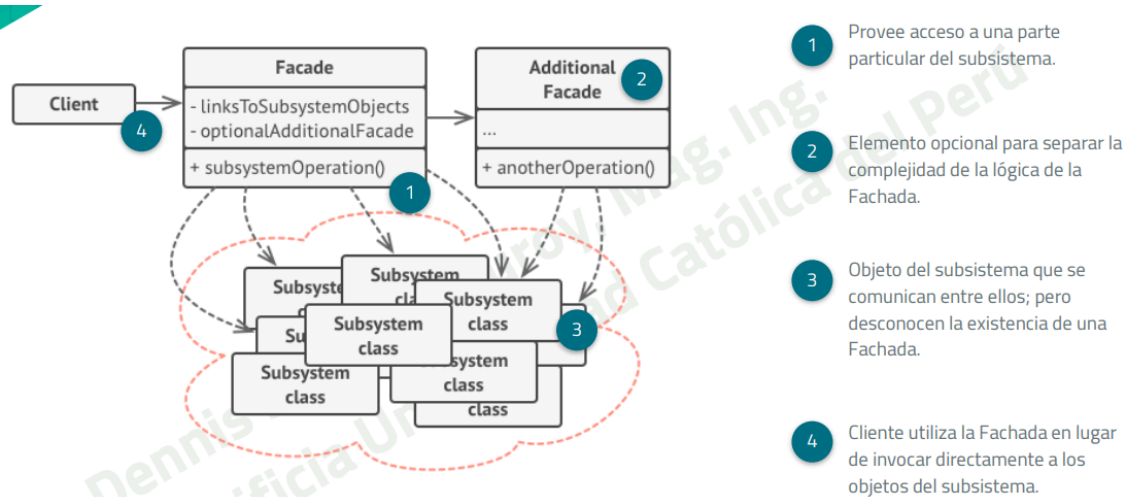
Cuándo usarlo:

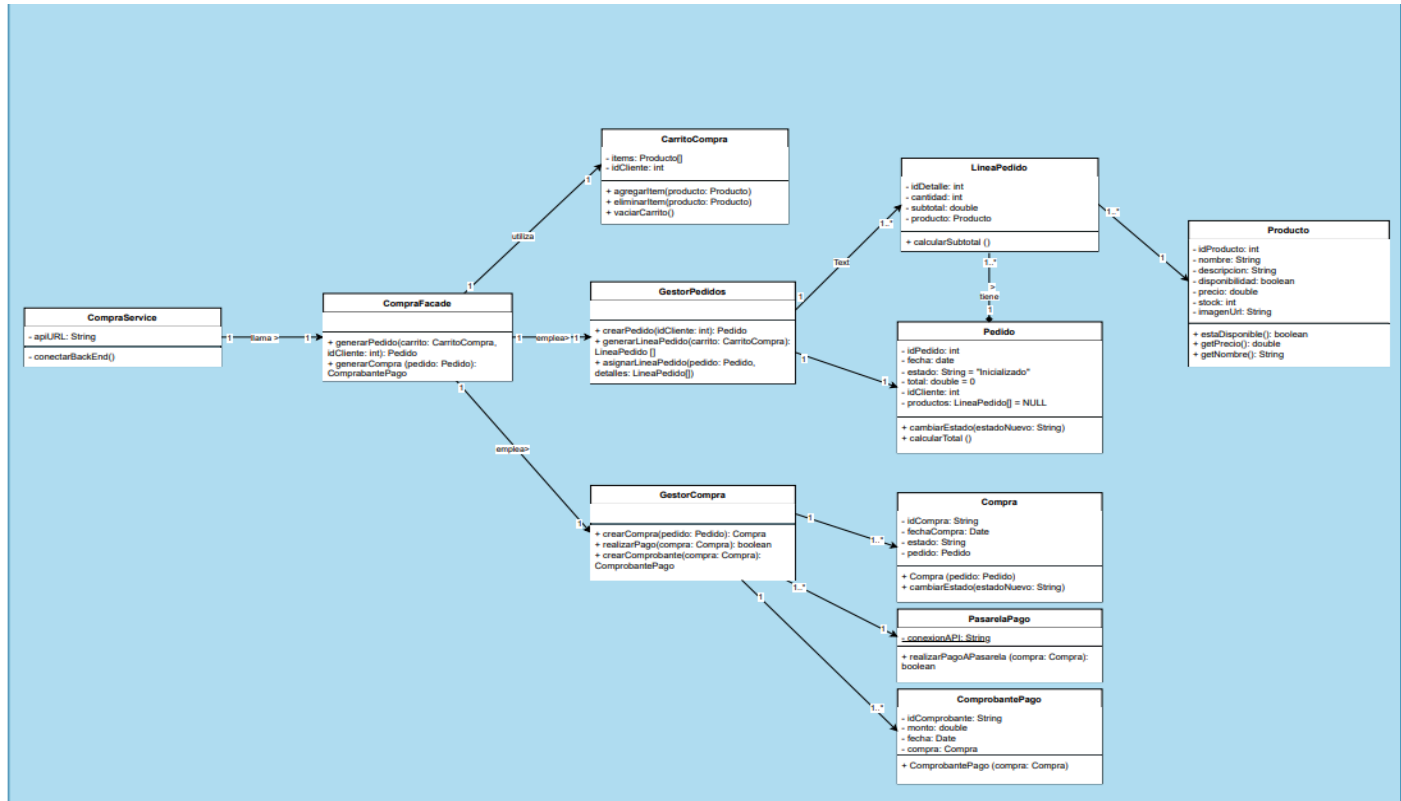
-  Añadir funcionalidades en tiempo de ejecución
-  Evitar explosión de subclases
-  Combinar comportamientos flexiblemente

Consecuencias:

- + Flexibilidad extrema
- + Cumple Open/Closed Principle
- — Complejo remover decoradores específicos
- — Orden de decoradores afecta resultado

3. Facade (Fachada)





Problema que resuelve: Simplificar interfaces complejas de subsistemas.

Cómo funciona: Provee una interfaz simple que oculta la complejidad del subsistema.

Ejemplo:

java

```

// Subsistema complejo
class SistemaVideo {
    public void configurar() { ... }
    public void reproducir() { ... }
    public void detener() { ... }
}

class SistemaAudio {
    public void ajustarVolumen() { ... }
    public void configurarCodec() { ... }
}

class SistemaDatos {
    public void cargarArchivo() { ... }
    public void verificarFormato() { ... }
}

// Fachada simplificada
class ReproductorMultimedia {
    private SistemaVideo video;
    private SistemaAudio audio;
    private SistemaDatos datos;
}
  
```

```

    public void reproducirPelicula(String archivo) {
        datos.cargarArchivo(archivo);
        datos.verificarFormato();
        video.configurar();
        audio.configurarCodec();
        audio.ajustarVolumen();
        video.reproducir();
    }
}

// Cliente usa solo la fachada
ReproductorMultimedia reproductor = new ReproductorMultimedia();

reproductor.reproducirPelicula("pelicula.mp4");

```

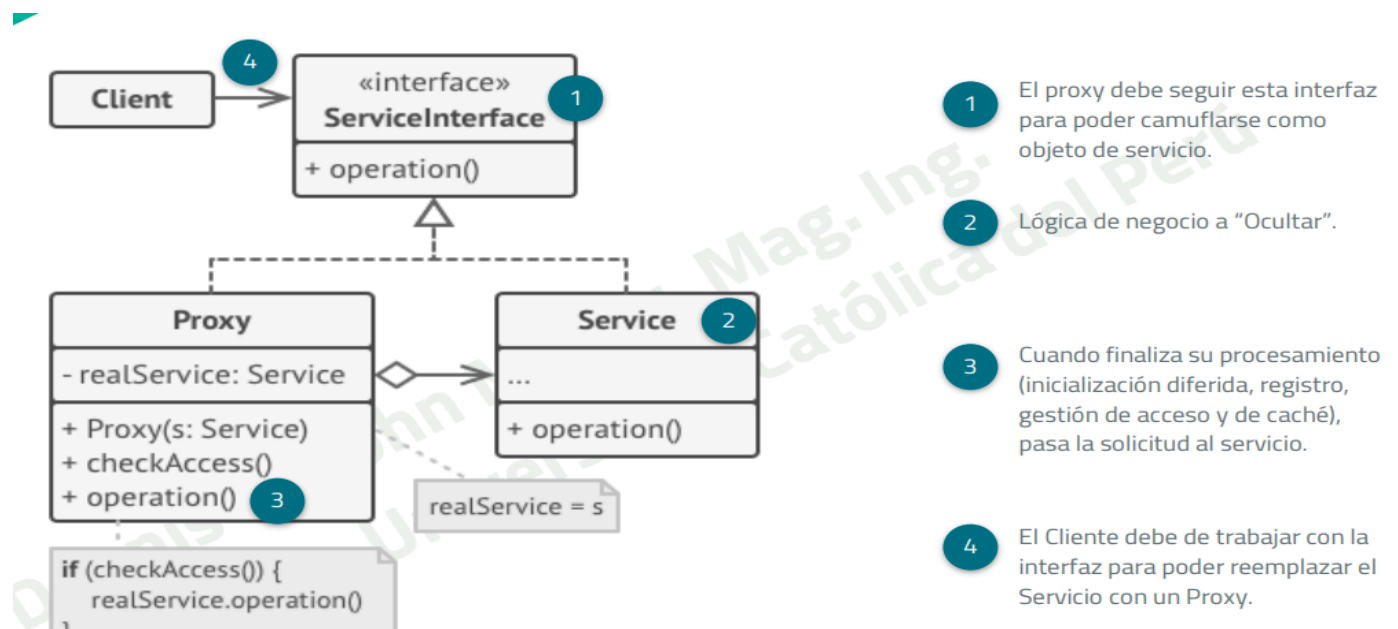
Cuándo usarlo:

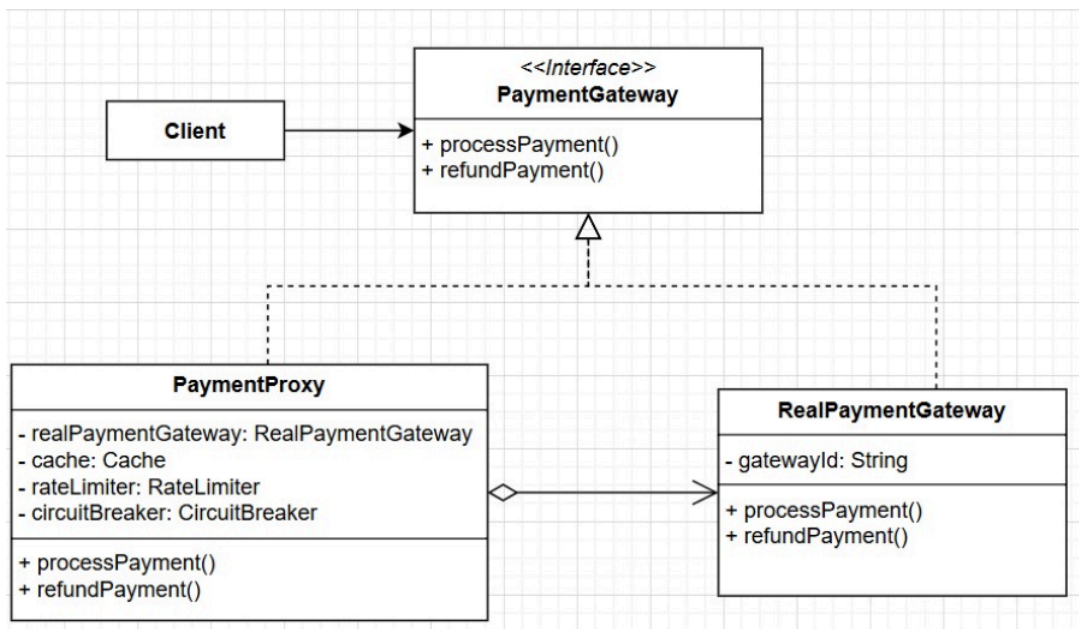
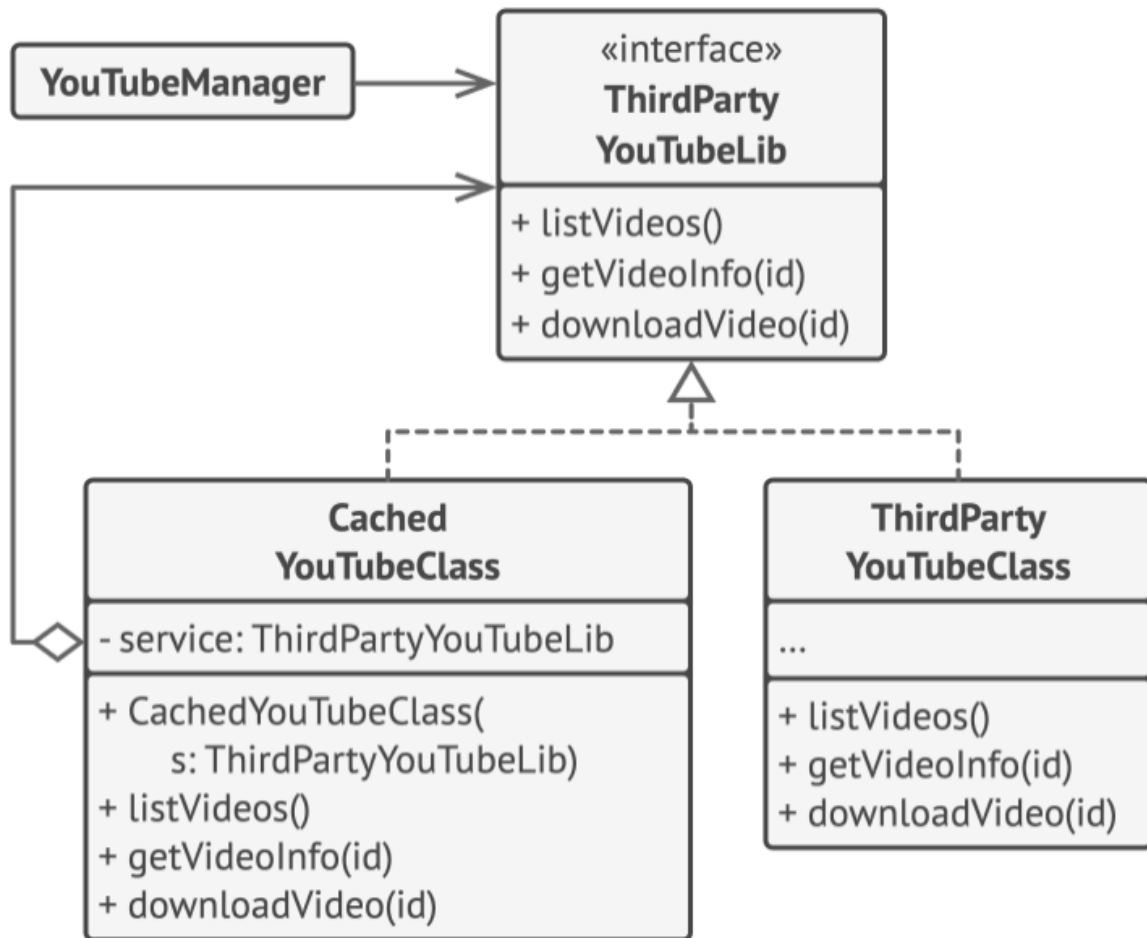
- ☒ Simplificar subsistemas complejos
- ☒ Desacoplar cliente de implementación
- ☒ Crear puntos de entrada a librerías

Consecuencias:

- ☒ Aísla complejidad del cliente
- ☒ Facilita mantenimiento
- ☒ Puede convertirse en "God Object"

4. Proxy





Problema que resuelve: Controlar acceso a objetos, añadiendo lógica adicional.

Cómo funciona: Crea un sustituto que controla el acceso al objeto real.

Ejemplo:

```
java

interface Imagen {
    void mostrar();
}

class ImagenReal implements Imagen {
    private String archivo;

    public ImagenReal(String archivo) {
        this.archivo = archivo;
        cargarDesdeDisco(); // Operación pesada
    }

    public void mostrar() {
        System.out.println("Mostrando: " + archivo);
    }

    private void cargarDesdeDisco() {
        System.out.println("Cargando imagen: " + archivo);
    }
}

class ProxyImagen implements Imagen {
    private ImagenReal imagenReal;
    private String archivo;

    public ProxyImagen(String archivo) {
        this.archivo = archivo;
    }

    public void mostrar() {
        if (imagenReal == null) {
            imagenReal = new ImagenReal(archivo); // Carga diferida
        }
        imagenReal.mostrar();
    }
}

// Uso - la imagen se carga solo cuando se necesita
Imagen imagen1 = new ProxyImagen("foto1.jpg");
Imagen imagen2 = new ProxyImagen("foto2.jpg");

// La imagen no se ha cargado aún...
```

```
imagen1.mostrar(); // Ahora sí se carga y muestra
```

Tipos de Proxy:

- Virtual Proxy: Carga diferida de objetos pesados
- Protection Proxy: Control de acceso
- Remote Proxy: Comunicación con objetos remotos
- Caching Proxy: Almacenamiento en caché

Cuándo usarlo:

-  Carga diferida (Lazy Loading)
-  Control de acceso
-  Caché de resultados
-  Logging y monitoreo

Consecuencias:

-  Control transparente
-  Funciona con servicios no disponibles
-  Aumenta latencia
-  Mayor complejidad

Guía Rápida de Selección

Patrón	Usar cuando...	Ejemplo común
Adapter	Interfaces incompatibles	Integrar APIs externas
Decorator	Funcionalidades dinámicas	Sistema de permisos, notificaciones
Facade	Subsistemas complejos	APIs simplificadas de librerías
Proxy	Control de acceso/carga	Lazy loading, cache, seguridad

Principios de Diseño Aplicados

- Adapter: Abierto/Cerrado - nuevas adaptaciones sin modificar cliente
- Decorator: Responsabilidad Única - cada decorador una función
- Facade: Principio de Mínimo Conocimiento - cliente conoce solo fachada
- Proxy: Segregación de Interfaces - cliente usa interfaz, no implementación

Recomendación: Estos patrones son esenciales para crear sistemas mantenibles y extensibles, especialmente cuando trabajas con código legacy o integraciones complejas.